```
//1(A)
(MUX)module
mux(d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2,out);
input d0,d1,d2,d3,d4,d5,d6,d7,s0,s1,s2;
output out;
wire t1,t2,t3,t4,t5,t6,t7,t8,t9,t10,t11;
not(t1,s0);
not(t2,s1);
not(t3,s2);
and(t4,d0,t1,t2,t3);
and(t5,d1,s0,t2,t3);
and(t6,d2,t1,s1,t3);
and(t7,d3,s0,s1,t3);
and(t8,d4,t1,t2,s2);
and(t9,d5,s0,t2,s2);
and(t10,d6,t1,s1,s2);
and(t11,d7,s0,s1,s2)
or(out,t4,t5,t6,t7,t8,t9,t10,t11);
endmodule

(DEMUX) module
demultiplexer(d,s0,s1,s2,d0,d1,d2,d3,d4,d5,d6,d7);
input d,s0,s1,s2;
output d0,d1,d2,d3,d4,d5,d6,d7;
wire t1,t2,t3;
not(t1,s0);
not(t2,s1);
not(t3,s2);
and(d0,d,t1,t2,t3);
and(d1,d,s0,t2,t3);
and(d2,d,t1,s1,t3);
and(d3,d,s0,s1,t3);
and(d4,d,t1,t2,s2);
and(d5,d,s0,t2,s2);
and(d6,d,t1,s1,s2);
and(d7,d,s0,s1,s2);
endmodule

(ENCODER)module encoder8to3(din,a,b,c);
input [0:7] din;
output a,b,c;
reg a,b,c;
always @(din)
```

```
//(2)
(JK FF)
module jk(q,qb,clk,rst,j,k); input clk,rst,j,k;
output reg q,qb;
always @(negedge clk)
begin
   if(rst == 1'b1)
   begin
      q = 1'b0;qb = 1'b1;
   end
   else
   begin
      case({j,k})
      2'b00:
      begin
         q = q;qb = !q;
      end
      2'b01:
      begin
         q = 1'b0;qb = 1'b1;
      end
      2'b10:
      begin
         q = 1'b1;qb = 1'b0;
      end
      2'b11:
      begin
         q = !q;qb = !q;
      end
      endcase
   end
end
endmodule

(4BIT  BC)
module count(c,cb,clk,rst,J,K);
output [3:0] c,cb;input clk,rst,J,K;
wire t1,t2;assign t1 = c[0] & c[1];
assign t2 = t1 & c[2];JK a1(c[0],cb[0],clk,rst,J,K);
JK a2(c[1],cb[1],clk,rst,c[0],c[0]);
JK a3(c[2],cb[2],clk,rst,t1,t1);
JK a4(c[3],cb[3],clk,rst,t2,t2);
Endmodule (2)//
```

```
//(3)module tlc(clk,reset,p1,p2,p3,p4,p5);input clk,reset;
output reg [4:0] p1,p2,p3,p4;output reg [3:0] p5;
reg [5:0] Sig;always @(posedge clk)
begin
   if(reset == 0)
   begin
      Sig <= 0;p1 <= 5'b00100;p2 <= 5'b00100;
      p3 <= 5'b00100; p4 <= 5'b00100;p5 <= 4'b1111;
   end
   else
   begin
      Sig <= Sig + 1;
      case(Sig)
      0: begin
         p1<=5'b11001; p2<=5'b00100;
         p3<=5'b00100; p4<=5'b00100;p5<=4'b1111;
      end
4: begin
         p1<=5'b01000; p2<=5'b00100;
         p3<=5'b00100; p4<=5'b00100;
      end
      8: begin
         p1<=5'b00100; p2<=5'b11001;
         p3<=5'b00100; p4<=5'b00100;
      end
      12: begin
         p2<=5'b01000;
      end
16: begin
         p2<=5'b00100; p3<=5'b11001;
      end
      20: begin
         p3<=5'b01000;
      end
24: begin
         p3<=5'b00100; p4<=5'b11001;
      end
      28: begin
         p4<=5'b01000;
      end
32: begin
         p4<=5'b00100; p5<=4'b0000;
      end
      36: begin
         Sig<=0;
      end
      endcase
   end
end
endmodule (3)//
```

```
//1(B)
(HA) module ha(a,b,s,c);
input a,b;
output s,c;
assign s = a ^ b;
assign c = a & b;
endmodule

(FA) module fa(a,b,c,s,d);
input a,b,c;
output s,d;
assign s = a ^ b ^ c;
assign d = (a & b) | (b & c) | (c & a);
endmodule

(SBA) module SBA(a,b,sum,carry,Cin);
input [3:0] a,b;
input Cin;
output [3:0] sum;
output carry;
wire [2:0] c;
fa u0(a[0],b[0],Cin,sum[0],c[0]);
fa u1(a[1],b[1],c[0],sum[1],c[1]);
fa u2(a[2],b[2],c[1],sum[2],c[2]);
fa u3(a[3],b[3],c[2],sum[3],carry);
endmodule

(CLAD) module cla_adder(
   input [3:0] A,B,
   input cin,
   output [3:0] S,
   output cout
);
wire [3:0] P,G;
wire [4:0] C;
assign P = A ^ B;
assign G = A & B;
cla_block gen_c(P,G,cin,C);
assign S = P ^ C[3:0];
assign cout = C[4];
endmodule
module cla_block(
   input [3:0] P,G
```

```
//(7) (ALU/MAC) SORCE CODE
module alu (
   input clk,
   input sel_load,
   input [3:0] A, B,
   input [3:0] ALU_Sel_in,
   output [3:0] ALU_Out,
   output CarryOut
);
reg [3:0] ALU_Result;reg [3:0] ALU_Sel;
wire [4:0] tmp;assign tmp = {1'b0, A} + {1'b0, B};
assign CarryOut = tmp[4];assign ALU_Out =
ALU_Result;
always @(posedge clk)
begin
   if (sel_load)
   begin
      ALU_Sel <= ALU_Sel_in;
   end
end
always @(*)
begin
   case (ALU_Sel)
      4'b0000: ALU_Result = A + B;
      4'b0001: ALU_Result = A - B;
      4'b0010: ALU_Result = A * B;
      4'b0011: ALU_Result = (B != 0) ? A / B : 4'b0;
      4'b0100: ALU_Result = A << 1;
      4'b0101: ALU_Result = A >> 1;
      4'b0110: ALU_Result = {A[2:0], A[3]};
      4'b0111: ALU_Result = {A[0], A[3:1]};
      4'b1000: ALU_Result = A & B;
      4'b1001: ALU_Result = A | B;
      4'b1010: ALU_Result = A ^ B;
      4'b1011: ALU_Result = ~(A | B);
      4'b1100: ALU_Result = ~(A & B);
      4'b1101: ALU_Result = ~(A ^ B);
      4'b1110: ALU_Result = (A > B) ? 4'd1 : 4'd0;
      4'b1111: ALU_Result = (A == B) ? 4'd1 : 4'd0;
      default: ALU_Result = 4'd0;
   endcase
end
endmodule
```

```
//(12)(LCD RL78)
#include "LCD.h"
void main(void)
{
   char t[16] = "Temp: 00.0";unsigned long int a;
   unsigned long int b;InitializeLCD();
   CursorOFF();DisplayLCD(1, "Room
Temperature");R_ADC_Set_OperationOn();
   while(1U)
   {
      ADCS = 1U; while(ADCS == 0);
      a = (ADCR >> 6U);a = (a * 1400) / 1023;
      t[6] = a / 100 + '0';a -= (a / 100) * 100;
      t[7] = a / 10 + '0';t[9] = a % 10 + '0';
      DisplayLCD(2, t);for(b = 0; b < 0x1FFFF; b++);
   }

} (12)//

//(13) (TIM/COUNT RL 78)
#include "LCD.h"#include <stdint.h>
unsigned long int Frequency;
extern volatile uint32_t g_tau0_ch7_width;
void main(void)
{
   char t[25] = "Frequency: ";unsigned long int a;
   InitializeLCD();DisplayLCD(1, "Freq. Counter:");
   EI(); while(1)
   {
      if(g_tau0_ch7_width != 0)
      {
         Frequency = 635000 / g_tau0_ch7_width;
         t[10] = (Frequency / 100000) % 10 + '0';
         t[11] = (Frequency / 10000) % 10 + '0';
         t[12] = (Frequency / 1000) % 10 + '0';
         t[13] = (Frequency / 100) % 10 + '0';
         t[14] = (Frequency / 10) % 10 + '0';
         t[15] = Frequency % 10 + '0';
         DisplayLCD(2, t);
      }
      for(a = 0; a < 0x5FFFF; a++);
   }
} (13)//
```

```
//(5)(8B SI..MUL)
module multi_signed(prod,multiplicand,multiplier);
output [15:0] prod;
input signed [7:0] multiplicand;
input signed [7:0] multiplier;
wire signed [15:0] shift1,shift2,shift3,shift4;
wire signed [15:0] shift5,shift6,shift7,shift8;
wire signed [15:0] add1,add2,add3,add4;
wire signed [15:0] add5,add6,add7,add8;
wire signed [15:0] prod_s;
assign shift1 = {{8{multiplicand[7]}},multiplicand};
assign shift2 = shift1 << 1;assign shift3 = shift1 << 2;
assign shift4 = shift1 << 3;assign shift5 = shift1 << 4;
assign shift6 = shift1 << 5;assign shift7 = shift1 << 6;
assign shift8 = shift1 << 7;
assign add1 = (multiplier[0] == 1'b1) ? shift1 : 16'b0;
assign add2 = (multiplier[1] == 1'b1) ? shift2 : 16'b0;
assign add3 = (multiplier[2] == 1'b1) ? shift3 : 16'b0;
assign add4 = (multiplier[3] == 1'b1) ? shift4 : 16'b0;
assign add5 = (multiplier[4] == 1'b1) ? shift5 : 16'b0;
assign add6 = (multiplier[5] == 1'b1) ? shift6 : 16'b0;
assign add7 = (multiplier[6] == 1'b1) ? shift7 : 16'b0;
assign add8 = (multiplier[7] == 1'b1) ? shift8 : 16'b0;
assign prod_s = add1+add2+add3+add4+
          add5+add6+add7-add8;
assign prod = prod_s;
endmodule (5)//

//(11)(3 PHASE PWM)
void R_TAU0_Channel0_Start(void);
void R_TAU0_Channel1_Start(void);
void R_TAU0_Channel2_Start(void);

void main(void)
{
  R_TAU0_Channel0_Start();
  R_TAU0_Channel1_Start();
  R_TAU0_Channel2_Start();

  while(1);
}
(11)//
```

```
//(4)
(P. PA)module
parallel(clk,add,load,clr,data,cal,result);
input clk,clr,cal,load;
input [2:0] add;input [11:0] data;
output reg [11:0] result;reg [11:0] ram[7:0];
wire [11:0] tem;
always @(posedge clk)
begin
  if(~clr)
  begin
    ram[0]=12'b0; ram[1]=12'b0;ram[2]=12'b0;
    ram[3]=12'b0; ram[4]=12'b0;ram[5]=12'b0;
    ram[6]=12'b0; ram[7]=12'b0;
  end
  else if(~load)
    ram[add]=data;
end
assign tem = ram[0]+ram[1]+ram[2]+ram[3]+
        ram[4]+ram[5]+ram[6]+ram[7];
always @(posedge clk)
begin
  if(~load)
    result=data;
  else if(~cal)
    result=tem;
  else
    result=ram[add];
end
endmodule (4)//

//(9)(I/O)
void main(void)
{
  long a;
  R_MAIN_UserInit();
  while(1U)
  {
    P6.3 = 1; P6.2 = 1;
    for(a = 0; a <= 0x3FFFE; a++);P6.3 = 0;P6.2 = 0;
    for(a = 0; a <= 0x3FFFF; a++);
  }
} (9)//
```

```
Begin
 case(din)
   8'b10000000: begin a=1'b0; b=1'b0; c=1'b0; end
   8'b01000000: begin a=1'b0; b=1'b0; c=1'b1; end
   8'b00100000: begin a=1'b0; b=1'b1; c=1'b0; end
   8'b00010000: begin a=1'b0; b=1'b1; c=1'b1; end
   8'b00001000: begin a=1'b1; b=1'b0; c=1'b0; end
   8'b00000100: begin a=1'b1; b=1'b0; c=1'b1; end
   8'b00000010: begin a=1'b1; b=1'b1; c=1'b0; end
   8'b00000001: begin a=1'b1; b=1'b1; c=1'b1; end
   default: begin a=1'bz; b=1'bz; c=1'bz; end
   endcase
end
endmodule

(DECODER) module decoder3to8(in,out,en);
input [2:0] in;
input en;
output [7:0] out;
reg [7:0] out;
always @(in or en)
begin
  if(en)
  begin
    out = 8'd0;
    case(in)
    3'b000: out[0] = 1'b1;
    3'b001: out[1] = 1'b1;
    3'b010: out[2] = 1'b1;
    3'b011: out[3] = 1'b1;
    3'b100: out[4] = 1'b1;
    3'b101: out[5] = 1'b1;
    3'b110: out[6] = 1'b1;
    3'b111: out[7] = 1'b1;
    default: out = 8'd0;
    endcase
  end
  else
    out = 8'd0;
end
endmodule
1(A)//
```

```
//(10..A)(STE.MOT)
#include <htc.h>
#define Stepper PORTD
void main(void)
{
  long i;
  TRISD = 0x00;
  while(1)
  {
    Stepper = 0x16;
    for(i=0; i<=0x8000; i++);
    Stepper = 0x1C;
    for(i=0; i<=0x8000; i++);
    Stepper = 0x19;
    for(i=0; i<=0x8000; i++);
    Stepper = 0x13;
    for(i=0; i<=0x8000; i++);
  }
} (10..A)//



//(10..B) (DC MOT PIC)
#include <htc.h>
#define MOTOR RD4
#define Duty_Cycle 80
void main(void)
{
  long i;
  TRISD = 0x00;
  while(1)
  {
    MOTOR = 1;
    for(i=0; i<Duty_Cycle; i++);
    MOTOR = 0;
    for(i=0; i<(100-Duty_Cycle); i++);
  }
}
(10..B)//
```

```
(CONSTRAINT CODE) ## CLOCK
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
## SELECTION LOAD BUTTON
set_property PACKAGE_PIN U18 [get_ports sel_load]
set_property IOSTANDARD LVCMOS33 [get_ports
sel_load]
## INPUT A
set_property PACKAGE_PIN V17 [get_ports {A[0]}]
set_property PACKAGE_PIN V16 [get_ports {A[1]}]
set_property PACKAGE_PIN W16 [get_ports {A[2]}]
set_property PACKAGE_PIN W17 [get_ports {A[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports
{A[*]}]
## INPUT B
set_property PACKAGE_PIN W15 [get_ports {B[0]}]
set_property PACKAGE_PIN V15 [get_ports {B[1]}]
set_property PACKAGE_PIN W14 [get_ports {B[2]}]
set_property PACKAGE_PIN W13 [get_ports {B[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports
{B[*]}]
## ALU SELECT INPUT
set_property PACKAGE_PIN V2 [get_ports
{ALU_Sel_in[0]}]
set_property PACKAGE_PIN T3 [get_ports
{ALU_Sel_in[1]}]
set_property PACKAGE_PIN T2 [get_ports
{ALU_Sel_in[2]}]
set_property PACKAGE_PIN R3 [get_ports
{ALU_Sel_in[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports
{ALU_Sel_in[*]}]
## ALU OUTPUT
set_property PACKAGE_PIN U16 [get_ports
{ALU_Out[0]}]
set_property PACKAGE_PIN E19 [get_ports
{ALU_Out[1]}]
set_property PACKAGE_PIN U19 [get_ports {ALU_Out[2]}]
set_property PACKAGE_PIN V19 [get_ports {ALU_Out[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports
{ALU_Out[*]}]
## CARRY OUTPUT
set_property PACKAGE_PIN W18 [get_ports CarryOut]
set_property IOSTANDARD LVCMOS33 [get_ports
CarryOut]
(7)//
```

```
input cin,
  output [4:0] C
);
assign C[0] = cin;
assign C[1] = G[0] | (P[0] & cin);
assign C[2] = G[1] |
          (P[1] & G[0]) |
          (P[1] & P[0] & cin);
assign C[3] = G[2] |
          (P[2] & G[1]) |
          (P[2] & P[1] & G[0]) |
          (P[2] & P[1] & P[0] & cin);
assign C[4] = G[3] |
          (P[3] & G[2]) |
          (P[3] & P[2] & G[1]) |
          (P[3] & P[2] & P[1] & G[0]) |
          (P[3] & P[2] & P[1] & P[0] & cin);
Endmodule 1(B)//

//1(C)
(EXCESS 3) module bcdtoexcess3(input
D0,D1,D2,D3,
          output E0,E1,E2,E3);

assign E3 = D3 | (D2 & D0) | (D2 & D1);
assign E2 = (D2 & (~D1) & (~D0)) |
        ((~D2) & D0) |
        ((~D2) & D1);
assign E1 = ((~D1) & (~D0)) | (D1 & D0);
assign E0 = ~D0;
endmodule
1(C)//
```

```
//(14) (WATCHDOG RL78)
#define Firstswitch P6.0
#define Secondswitch P6.1
#define FirstLED P1.0
#define SecondLED P1.7
unsigned char b[15] = {
   0x01,0x02,0x04,0x08,0x10,
   0x20,0x40,0x80,0x40,0x20,
   0x10,0x08,0x04,0x02,0x01
};
void R_WDT_Create(void);
void R_WDT_Restart(void);
void main(void)
{
   unsigned char a,c;
   long d; a=0;c=1;R_WDT_Create();
   while(c)
   {
      P1=b[a];
      a++;
      if(a>=15)
         a=0;
      if(Firstswitch==0)
      {
         c=0;while(Firstswitch==0);
      }
      R_WDT_Restart();
      for(d=0;d<0x1FFFF;d++);
   }
   if(c==0)
      R_WDT_Restart();
   if(Secondswitch==0)
   {
      SecondLED=1; c=1;P1=0;c=0;
      R_WDT_Create();
      while(1)
      {
         FirstLED ^= 1;for(d=0;d<0xFFFF;d++);
      }
   }
}
(14)//
```

```
///(8) (SR FF)
(SOURCE)
module sr_ff(
   input clk,input rst,input S,R,output reg Q,
   output Qbar
);
always @(posedge clk or posedge rst)
begin
   if(rst)
   begin
      Q <= 1'b0;
   end
   else
   begin
      case({S,R})
      2'b00: Q <= Q;2'b01: Q <= 1'b0;
      2'b10: Q <= 1'b1;2'b11: Q <= 1'bx;
      endcase
   end
end
assign Qbar = ~Q;
endmodule

(CONSTRAINT)

## Clock (100 MHz)
set_property -dict {PACKAGE_PIN W5
IOSTANDARD LVCMOS33} [get_ports {clk}]
create_clock -period 10.00 -name sys_clk_pin -
waveform {0 5} [get_ports {clk}]
## Reset Button
set_property -dict {PACKAGE_PIN T18
IOSTANDARD LVCMOS33} [get_ports {rst}]
## Switches for S and R
set_property -dict {PACKAGE_PIN V17
IOSTANDARD LVCMOS33} [get_ports {S}]
set_property -dict {PACKAGE_PIN V16
IOSTANDARD LVCMOS33} [get_ports {R}]
## LEDs for Q and Qbar
set_property -dict {PACKAGE_PIN U16
IOSTANDARD LVCMOS33} [get_ports {Q}]
set_property -dict {PACKAGE_PIN U19
IOSTANDARD LVCMOS33} [get_ports {Qbar}]
```

```
//(6)
(SOURCE)
module led_blink(
   input wire clk,
   output reg [15:0] led
);

parameter CLK_FREQ = 100_000_000;
parameter BLINK_FREQ = 1;

localparam integer COUNT_MAX =
CLK_FREQ/(2*BLINK_FREQ);

reg [31:0] counter = 0;

initial led = 16'b0;

always @(posedge clk)
begin
   if(counter >= COUNT_MAX-1)
   begin
      counter <= 0;
      led <= ~led;
   end
   else
      counter <= counter+1;
end

endmodule
```

```
//(15) (ADC INTERFACE ARM 7)

#include <lpc214x.h>
#include "lcd.h"
#include "adclcd.h"
#include "adcseg.h"
#define led_DIR IO0DIR
#define led_SET IO0SET
#define led_CLR IO0CLR
#define swt_DIR IO1DIR
#define swt_SET IO1SET
#define swt_CLR IO1CLR
#define l8 (1<<30)
#define l9 (1<<31)
#define led_pin 0xC0000000
#define swt_pin 0x01800000
#define sw3 (1<<23)
#define sw4 (1<<24)
unsigned int a=0,b=0;
int getpinstate1(int pinnumber);
int main(void)
{
   led_DIR |= led_pin;
   swt_DIR &= ~(swt_pin);
   lcdinit();
   while(1)
   {
      if(!getpinstate1(sw3) && !getpinstate1(sw4))
      {
         led_CLR = l8 | l9;
         a=3;
         if(a!=b)
         {
            lcdinit();
            b=a;
         }
         lcddisplay(0x82,"ENIXS
TECHNOLOGY",16);
         lcddisplay(0xC3,"ADC-0809 DEMO",13);
         lcddisplay(0x94,"SW3-ADC Result:LCD",18);
         lcddisplay(0xD4,"SW4-ADC
```

```
//(8) (JK FF)
(SOURCE)
module jk_ff(
   input clk,input rst,input J,K,output reg Q,
   output Qbar
);
assign Qbar = ~Q;
always @(posedge clk or posedge rst)
begin
   if(rst)
   begin
      Q <= 1'b0;
   end
   else
   begin
      case({J,K})
      2'b00: Q <= Q;2'b01: Q <= 1'b0;
      2'b10: Q <= 1'b1;2'b11: Q <= ~Q;
      endcase
   end
end
endmodule
(CONSTRAINT)

## Clock (100 MHz)
set_property -dict {PACKAGE_PIN W5
IOSTANDARD LVCMOS33} [get_ports {clk}]
create_clock -period 10.00 -name sys_clk_pin -
waveform {0 5} [get_ports {clk}]
## Reset Button (BTNU)
set_property -dict {PACKAGE_PIN T18
IOSTANDARD LVCMOS33} [get_ports {rst}]
## Inputs J and K (Switches SW0, SW1)
set_property -dict {PACKAGE_PIN V17
IOSTANDARD LVCMOS33} [get_ports {J}]
set_property -dict {PACKAGE_PIN V16
IOSTANDARD LVCMOS33} [get_ports {K}]
## Outputs Q and Qbar (LEDs LD0, LD1)
set_property -dict {PACKAGE_PIN U16
IOSTANDARD LVCMOS33} [get_ports {Q}]
set_property -dict {PACKAGE_PIN U19
IOSTANDARD LVCMOS33} [get_ports {Qbar}]
```

(CONSTRAINT)

```
## Clock Signal
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports
clk]
create_clock -period 10.00 [get_ports clk]

## LED Connections
set_property PACKAGE_PIN U16 [get_ports {led[0]}]
set_property PACKAGE_PIN E19 [get_ports {led[1]}]
set_property PACKAGE_PIN U19 [get_ports {led[2]}]
set_property PACKAGE_PIN V19 [get_ports {led[3]}]
set_property PACKAGE_PIN W18 [get_ports
{led[4]}]
set_property PACKAGE_PIN U15 [get_ports {led[5]}]
set_property PACKAGE_PIN V14 [get_ports {led[6]}]
set_property PACKAGE_PIN U14 [get_ports {led[7]}]
set_property PACKAGE_PIN V13 [get_ports {led[8]}]
set_property PACKAGE_PIN V3 [get_ports {led[9]}]
set_property PACKAGE_PIN W3 [get_ports
{led[10]}]
set_property PACKAGE_PIN U3 [get_ports {led[11]}]
set_property PACKAGE_PIN P3 [get_ports {led[12]}]
set_property PACKAGE_PIN N3 [get_ports {led[13]}]
set_property PACKAGE_PIN P1 [get_ports {led[14]}]
set_property PACKAGE_PIN L1 [get_ports {led[15]}]

## IOSTANDARD
set_property IOSTANDARD LVCMOS33 [get_ports
{led[*]}]
```

(6)//

(D FF)
(SOURCE)

```
module d_ff(
    input clk,input rst,input D,output reg Q,
    output Qbar
);
assign Qbar = ~Q;
always @(posedge clk or posedge rst)
begin
    if(rst)
    begin
        Q <= 1'b0;
    end
    else
    begin
        Q <= D;
    end
end
endmodule
```

(CONSTRAINT)

```
## Clock (100 MHz)
set_property -dict {PACKAGE_PIN W5
IOSTANDARD LVCMOS33} [get_ports {clk}]
create_clock -period 10.00 -name sys_clk_pin -
waveform {0 5} [get_ports {clk}]

## Reset Button (BTNU)
set_property -dict {PACKAGE_PIN T18
IOSTANDARD LVCMOS33} [get_ports {rst}]

## Input D (Switch SW0)
set_property -dict {PACKAGE_PIN V17
IOSTANDARD LVCMOS33} [get_ports {D}]

## Outputs Q and Qbar (LEDs LD0 and LD1)
set_property -dict {PACKAGE_PIN U16
IOSTANDARD LVCMOS33} [get_ports {Q}]
set_property -dict {PACKAGE_PIN U19
IOSTANDARD LVCMOS33} [get_ports {Qbar}]
```

(8)//

(T FF) (SOURCE)

```
module t_ff(
    input clk,input rst,input T,output reg Q,
    output Qbar
);
assign Qbar = ~Q;
always @(posedge clk or posedge rst)
begin
    if(rst)
    begin
        Q <= 1'b0;
    end
    else
    begin
        if(T)
            Q <= ~Q;
        else
            Q <= Q;
    end
end
endmodule
```

(CONSTRAINT)

```
## Clock (100 MHz onboard oscillator)
set_property -dict {PACKAGE_PIN W5
IOSTANDARD LVCMOS33} [get_ports {clk}]
create_clock -period 10.00 -name sys_clk_pin -
waveform {0 5} [get_ports {clk}]
## Reset button (BTNU = T18)
set_property -dict {PACKAGE_PIN T18
IOSTANDARD LVCMOS33} [get_ports {rst}]

## Input T from switch SW0
set_property -dict {PACKAGE_PIN V17
IOSTANDARD LVCMOS33} [get_ports {T}]

## Outputs Q and Qbar to LEDs LD0, LD1
set_property -dict {PACKAGE_PIN U16
IOSTANDARD LVCMOS33} [get_ports {Q}]
set_property -dict {PACKAGE_PIN U19
IOSTANDARD LVCMOS33} [get_ports {Qbar}]
```

(8)///

```
Result:SEG",18);
        }
        if(getpinstate1(sw3) && !getpinstate1(sw4))
        {
            led_CLR = led_pin;
            led_SET = l8;
            led_CLR = l9;
            a=1;
            if(a!=b)
            {
                lcdinit();
                b=a;
            }
            adclcd();
        }
        if(!getpinstate1(sw3) && getpinstate1(sw4))
        {
            led_CLR = led_pin;
            led_SET = l9;
            led_CLR = l8;
            a=2;
            if(a!=b)
            {
                lcdinit();
                b=a;
                lcddisplay(0x80,"Multichannel ADC
CH0",20);
                lcddisplay(0xC2,"Toggle Swt to",13);
                lcddisplay(0x98,"7SEG Mode",9);
            }
            adcseg();
        }
    }
}
int getpinstate1(int pinnumber)
{
    int pinblockstate = IO1PIN;
    int pinstate = (pinblockstate & pinnumber);

    return pinstate;
}
```

(15)//